Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Cloud Computing Architecture

Semester project report

Group 033
Ling Zhang - 23-955-214
Xinge Xie - 23-943-046
Yuejiang Yu - 23-956-196

Systems Group
Department of Computer Science
ETH Zurich
April 14, 2025

Part 1 [25 points]

Using the instructions provided in the project description, run memcached alone (i.e., no interference), and with each iBench source of interference (cpu, l1d, l1i, l2, llc, membw). For Part 1, you must use the following `mcperf` command, which varies the target QPS from 5000 to 55000 in increments of 5000 (and has a warm-up time of 2 seconds with the addition of `-w 2`):

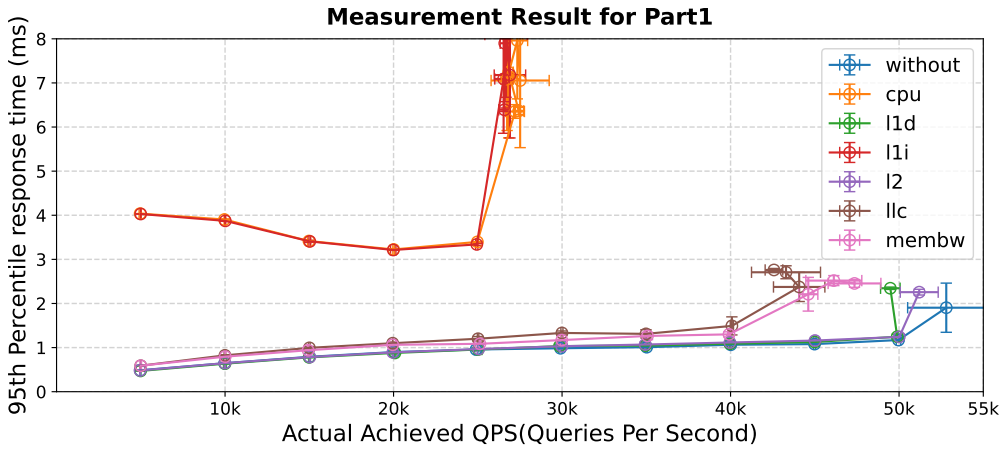
```
$ ./mcperf -s MEMCACHED_IP --loadonly
$ ./mcperf -s MEMCACHED_IP -a INTERNAL_AGENT_IP  \
           --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -w 2 -t 5 \
           --scan 5000:55000:5000
```

Repeat the run for each of the 7 configurations (without interference, and the 6 interference types) **at least 3 times** (3 should be sufficient in this case), and collect the performance measurements (i.e. the `client-measure` VM output). **Reminder:** after you have collected all the measurements, make sure you **delete your cluster**. Otherwise, you will easily use up the cloud credits. See the project description for instructions how to make sure your cluster is deleted.

(a) **[10 points]** Plot a line graph with the following stipulations:

- Queries per second (QPS) on the x-axis (the x-axis should range from 0 to 55K). (**note:** the actual achieved QPS, not the target QPS)
- 95th percentile latency on the y-axis (the y-axis should range from 0 to 8 ms).
- Label your axes.
- 7 lines, one for each configuration. Add a legend.
- State how many runs you averaged across and include error bars at each point in both dimensions.
- The readability of your plot will be part of your grade.

Answer:



The measurement result shows above. It was averaged across 3 runs and the error bars show the distribution of the results by mean and standard deviation. **P.S.** We stick to the above stipulation although the ranges for x-axis and y-axis are not large enough for a full view.

(b) **[6 points]** How is the tail latency and saturation point (the "knee in the curve") of memcached affected by each type of interference? What is your reasoning for these effects? Briefly describe your hypothesis.

Answer: First, an apparent observation is that under any types of interference, the tail latency increases and the saturaion points happen earlier compared with the ones without interference.

- `ibench-cpu`: Starting at 4ms in 5K QPS, the tail latency dips gradually to approximately 3.2ms at 25k QPS(saturation point). In fact, the dip is a bit unexpected but we try to propose our assumption below. Then, it increases drastically to around 7ms at 27k QPS and get a much larger variation around that level.

  Explanation: For the initial dip, it might be that CPU has adapted to the query burden and shifts to a higher performance state. Maybe there are more cache hit or CPU has better task scheduling with increasing queries. The surge at the saturation point might be that Memcached is a highly-computing-intensive application, which has a high demand on CPU.

- `ibench-l1d`: Interference on this and **l2** yeild similar performance which is equal to the one without interference. From 0.5ms at 5k QPS, the tail latency increases consistently to about 1.2ms at 50k QPS(saturation point). After that, the tail latency doubles to around 2.4ms and remains stable at this level. One thing interesting is that, after the saturation point, the actual achieved QPS becomes less than 50k, which also occurs in **llc**.

  Explanation: For the stable increase, we assume that it might be because the same data in *l1d* is not accessed very often, which means the L1 data cache cannot benefit from the locality with or without interference. And after the saturation point, the cache has too much cache misses to respond to much more queries. To explain the not achievable QPS, it might be that the L1 data cache is not large enough to fit so much queries.

- `ibench-l1i`: The change in tail latency of this type is almost the same as the one of **cpu**. A subtle difference is that the mean of final tail latency is about 0.1 ms higher.

  Explanation: The reason why there is a dip of tail latency before saturation point is that Memcached is desgined as a Key-Value store for small chunks of data and makes heavy use of the instruction cache instead of data cache. So after the "warm-up", benefitting from locality, the cache has much more warm hits, which speeds up the response significantly. And after the saturation point, the instruction cache may not large enough to store so much instructions, which leads to a severe rise of tail latency.

- `ibench-l2`: The change in tail latency of this type is almost the same as the one of l1d.

  Explanation: For the consistent increase, it might be because the same data in **l2** is not accessed very often as well, with or without interference. And after the saturation point, the cache has too much cache misses to respond to much more queries.

- `ibench-llc`: The tail latency increases gradually from 0.6ms at 5k QPS to 1.2ms at 40k QPS(saturation point). After that, it rises abruptly to about 2.3ms at 45k QPS.

  Explanation: Because L3 cache is the largest cache, there are more possibly reused data. Thus the interference would lead to a slightly worse tail latency than the one on **l1d** and **l2**. Also, Memcached requires a large global data structure to store the KV pairs. So after the saturation point, with the largest capacity, L3 cache is affected the most compared with L1 data cache and L2 cache.

- `ibench-membw`: The change in tail latency of this type is almost the same as the one of **llc**.

  Explanation: Although Memcached has a lot of random memory reads(queries) and writes(stores), it copes with small chucks of data. So the overall memory bandwidth requirement should not be high. Also, after the bandwidth is saturated, the tail latency increases correspondingly.

(c) **[2 points]**

- Explain the use of the `taskset` command in the container commands for memcached and iBench in the provided scripts.

  Answer: The **taskset** is a linux command used to set the CPU affinity for one or more running processes. More specifically, it can bind certain command to certain CPU. For example, "taskset -c 0 ./cpu 1200" binds the command "./cpu 1200" to CPU 0.
- Why do we run some of the iBench benchmarks on the same core as memcached and others on a different core? Give an explanation for each iBench benchmark.

  Answer: We note that CPU, L1d, L1i and L2 cache are binded to CPU 0 while the others are binded to CPU 1. It is reasonable for the former in that these four are designed to be integrated in a whole Core. As for the other two interferences, we assume that binding them to another CPU could minimize the introduced bias.

(d) **[2 points]** Assuming a service level objective (SLO) for memcached of up to 2 ms 95th percentile latency at 35K QPS, which iBench source of interference can safely be collocated with memcached without violating this SLO? Briefly explain your reasoning.

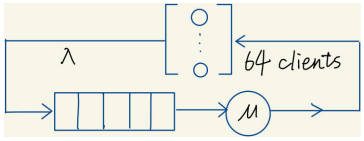
Answer: As shown in Figure 1, collocating interference of L1 data cache, L2 cache, L3 cache and memory bandwidth is safe to meet this SLO. Because the tail latency of these four interferences are about 1.2ms, which is only about a half of 2ms, at 35K QPS.

(e) **[5 points]** In the lectures you have seen queueing theory.

- Is the project experiment above an open system or a closed system? Explain why.

  Answer: It is an interactive, **closed** system because the **client-agent** VM sends request and waits for response before sending the next request in one connection.
- What is the number of clients in the system? How did you find this number?

  Answer: According to the command on **client-measure** VM and the help manual of **mcperf**, we know that "-T 16 -C 4" means there are 16 threads per connection and there are 4 connections. Thus, we have $16 \times 4 = 64$ clients in total during measuring.
- Sketch a diagram of the queueing system modeling the experiment above. Give a brief explanation of the sketch.



  Answer: The sketch is shown above. The circle, grid rectangle, top pattern, λ and μ mean CPU, queue, clients, request arrival rate and service rate.
- Provide an expression for the average response time of this system. Explain each term in this expression and match it to the parameters of the project experiment.

  Answer: We have average response time(R), client number(N), throughput(X), thinking time (Z, assumed 0 here). According to *the Interactive Law*, we have $R = \frac{N}{X} - Z$. Take a measurement without interference as an example. After the saturation point, the actual achieved QPS is 52996 s^{-1}. So we can compute the average response time $R = \frac{64}{52996\ s^{-1}} \approx 1.2076\ ms$. The discrepancy between this value and the practical value, 1.2477 ms, may be attributed to that the practical thinking time is greater than 0, and the formula misses the round-trip transmission time which should also be taken into account in practice.

Part 2 [31 points]

1. **Interference behavior [20 points]**

   (a) **[10.5 points]** Fill in the following table with the normalized execution time of each batch job with each source of interference. The execution time should be normalized to the job's execution time with no interference. Round the normalized execution time to 2 decimal places. Color-code each cell in the table as follows: **green** if the normalized execution time is less than or equal to 1.3, **orange** if the normalized execution time is over 1.3 and less or equal to 2, and **red** if the normalized execution time is greater than 2. The coloring of the first three cells is given as an example of how to use the cell coloring command. **Do not change the structure of the table. Only input the values inside the cells and color the cells properly.**

Workload	none	cpu	l1d	l1i	l2	llc	memBW
blackscholes	1.00	1.29	1.27	1.55	1.25	1.45	1.31
canneal	1.00	1.16	1.24	1.33	1.19	1.90	1.26
dedup	1.00	1.65	1.25	2.01	1.27	2.10	1.66
ferret	1.00	1.91	1.06	2.35	1.04	2.70	2.04
freqmine	1.00	2.00	1.03	2.02	1.02	1.88	1.60
radix	1.00	1.03	1.08	1.09	1.08	1.51	1.09
vips	1.00	1.68	1.63	1.80	1.63	1.92	1.65

   (b) **[7 points]** Explain what the interference profile table tells you about the resource requirements for each job. Give your reasoning behind these resource requirements based on the functionality of each job.

   Answer:

   - `blackscholes:`
     Explanation: This workload appears to be sensitive to L1 instruction cache interference (l1i), with a normalized execution time of 1.55, indicating a performance decrease when the L1i is contended. However, it seems to be less sensitive to other cache interferences and CPU contention.
   - `canneal:`
     Explanation: Canneal has a notable performance hit when there is memory bandwidth contention (memBW), showing a normalized execution time of 1.26. It indicates that canneal likely has high memory bandwidth usage, and its performance is degraded when the memory subsystem is stressed.
   - `dedup:`
     Explanation: Dedup shows significant sensitivity to L1 instruction cache (l1i) and last level (l3) cache (llc) contention, with times of 2.01 and 2.10 respectively. This suggests dedup may be an instruction-rich and data-intensive workload that also requires a considerable amount of memory bandwidth.
   - `ferret:`
     Explanation: Ferret has a very high sensitivity to L1 instruction cache interference (l1i) and last level cache (l3) interference with a normalized time of 2.35 and 2.70 respectively, indicating it relies heavily on the instruction cache and L3 cache. Its performance is also impacted by memory bandwidth interference, but to a lesser degree.

- `freqmine:`
  Explanation: This workload shows the greatest sensitivity to L1 instruction cache (l1i) interference with a normalized execution time of 2.02. It suggests freqmine's performance relies significantly on the efficient usage of the instruction cache.
- `radix:`
  Explanation: Radix exhibits very minor sensitivity to all types of interference tested. This indicates that radix either efficiently utilizes resources or its primary performance bottleneck lies elsewhere.
- `vips:`
  Explanation: Vips is notably sensitive to L1 instruction cache (l1i) and last level cache (llc) interference, with execution times of 1.80 and 1.92 respectively. It indicates that vips is likely computationally intensive and also requires significant cache inference in data and instructions. Notably, all the values are above 1.5, indicating that its performance bottleneck is quite wide-spread and even.

(c) **[2.5 points]** Which jobs (if any) seem like good candidates to collocate with memcached from Part 1, without violating the SLO of 2 ms P95 latency at 40K QPS? Explain why.

Answer:

The data shown in Table.1 suggests:

- **blackscholes**: Moderate impact on CPU and memBW. It could be a candidate for collocation, but caution is due to potential cumulative impacts.
- **canneal**: Shows minimal interference with CPU and memBW, making it a good candidate for collocation, despite its stress on LLC.
- **radix**: Minimal interference across both CPU and memBW, indicating it is an excellent candidate for collocation with Memcached.
- **dedup, ferret, and freqmine**: Significant interference in critical resources for Memcached, suggesting these are not suitable for collocation.
- **vips**: Moderate level of resource contention across all categories, making it less ideal for collocation with Memcached.

Given the data, radix remains the best choice for collocation with Memcached, as it has the least impact on CPU and memBW. Canneal also appears to be a good candidate, particularly because of its low impact on memBW.

2. **Parallel behavior [11 points]**

(a) **[7.5 points]** Plot a line graph with speedup as the y-axis (normalized time to the single thread config, Time_1 / Time_n) vs. number of threads on the x-axis (1, 2, 4 and 8 threads - see the project description for more details). Pay attention to the readability of your graph, it will be a part of your grade.

Answer: refer to figure 1 below.

(b) **[3.5 points]** Briefly discuss the scalability of each job: e.g., linear/sub-linear/super-linear. Do the applications gain significant speedup with the increased number of threads? Explain what you consider to be "significant".

Answer:

- `canneal:` Canneal's scalability is sub-linear, with diminishing returns as more threads are added. The initial speedup from 1 to 2 threads is significant, but after

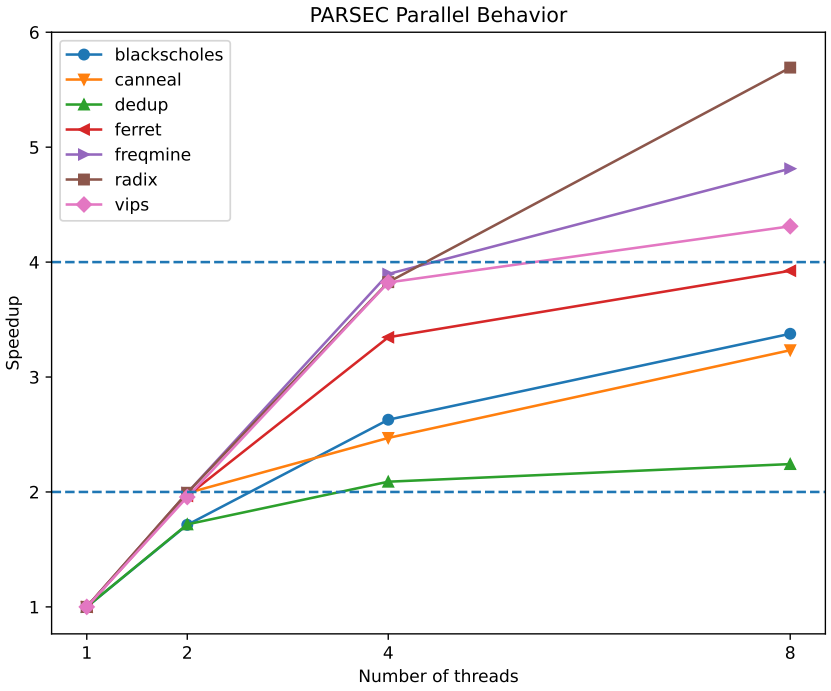


Figure 1: Measurment Result for Part2b

that, the increase in speedup flattens out, indicating that canneal may not benefit much from more than two threads.

- `dedup:` Dedup exhibits poor scalability with an almost flat line, suggesting that increasing the number of threads hardly improves performance. This indicates that dedup may have inherent sequential portions or contention that limits its parallel execution.
- `ferret:` Ferret's performance shows super-linear speedup from 1 to 4 threads, which is quite significant. However, the speedup from 4 to 8 threads is sub-linear, suggesting that it scales well up to a certain point before encountering diminishing returns.
- `freqmine:` Freqmine displays a super-linear speedup when increasing threads from 1 to 2 and maintains a strong linear trend thereafter. The speedup is significant across all thread increases, indicating excellent scalability.
- `radix:` Radix exhibits a super-linear speedup from 1 to 2 threads, and while the trend from 2 to 8 threads is linear, the speedup is less pronounced. However, considering the consistent increase, the speedup can be considered significant.
- `vips:` Vips shows a strong linear speedup across all thread increments, which is significant. The consistent upward trend indicates that vips scales well with the number of threads.

A "significant" speedup, in this context, refers to a speedup that clearly deviates from the baseline performance in a positive manner.

Part 3 [34 points]

1. **[17 points]** With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time [1] across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency $>$ 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
blackscholes	110.0	1.63
canneal	203.67	1.7
dedup	16.33	0.47
ferret	97.0	1.41
freqmine	129.0	0.0
radix	18.0	0.0
vips	40.67	3.09
total time	204.0	1.63

Answer:

- We compute the mean and standard deviation of the execution time across three runs as well as the total time to complete all jobs, as shown in the table above. The data is rounded to two decimal places. **Besides, it is worth noting that there are some 0s in *std* column. This is because the time format is only accurate to second and the running times are coincidentally the same.**
- As we can see from the figures 1, 2 and 3 below(in the next page), we did not violate the SLO requirement of 1ms p95 latency, and in fact there is still plenty of margin. So the SLO violation ratio for memcached for these three runs are 0/19, 0/18, 0/19.
- We also briefly describe the plotted figures here. Each figure consists of two subplots. The upper one is a bar plot, representing the p95 latency variation over time. The lower subplot shows the start, end, and duration of each job. We also indicate in parentheses which job runs on which node.

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. Use the colors proposed in this template (you can find them in `main.tex`). For example, use the vips color to annotate when

[1] Here, you should only consider the runtime, excluding time spans during which the container is paused.

vips started and stopped, the blackscholes color to annotate when blackscholes started and stopped etc.

Plots:

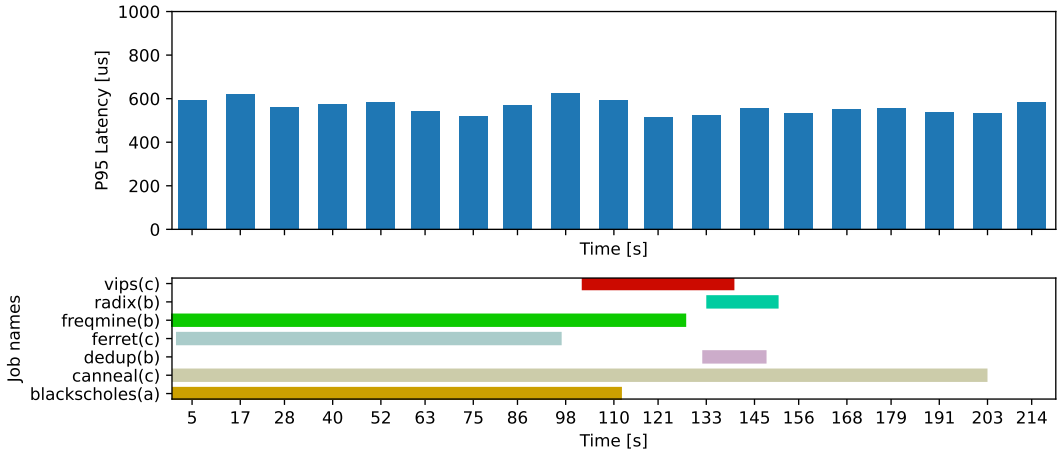


Figure 1: Result of the first run

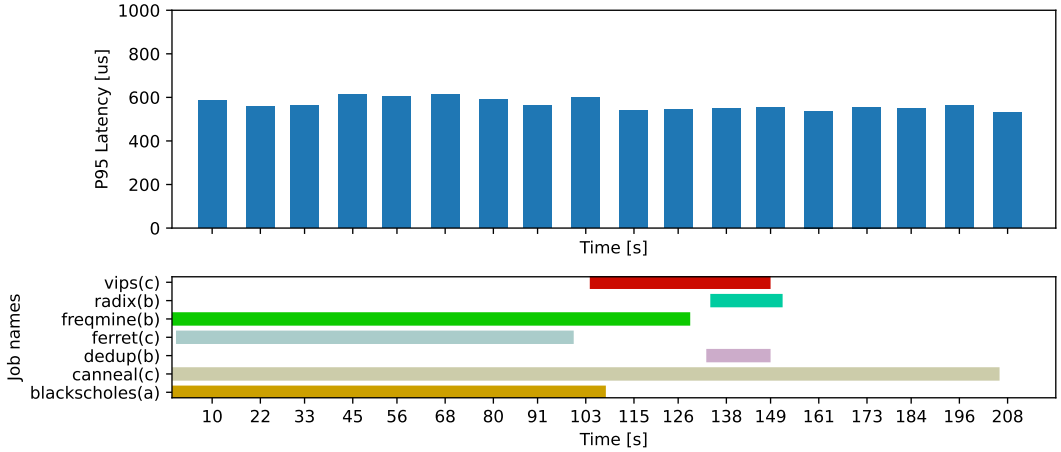


Figure 2: Result of the second run

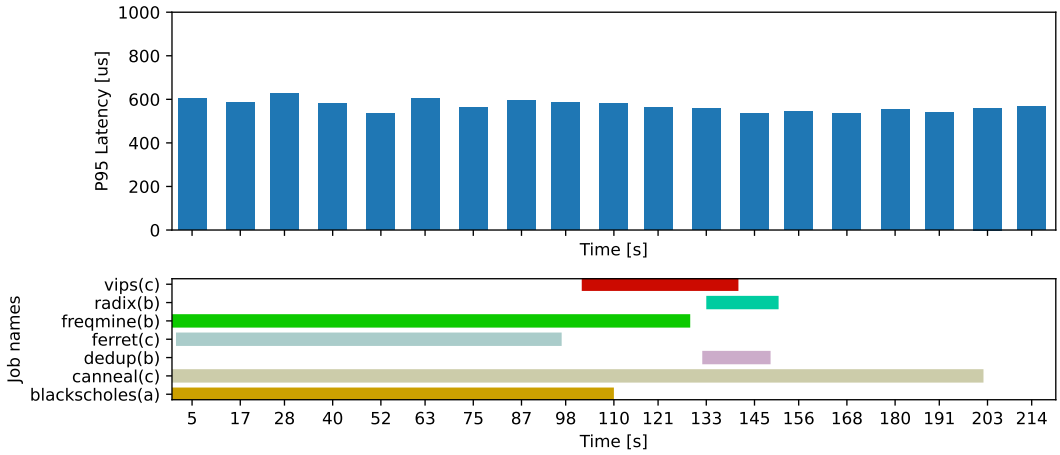


Figure 3: Result of the third run

2. [**17 points**] Describe and justify the "optimal" scheduling policy you have designed.

- Which node does memcached run on? Why?
  Answer:
  We run memcached on node-a-2core, which is of type n2d-highcpu-2. This is because from the part1, we know that memcached is a relatively lightweight system, which is mostly sensitive to CPU and L1i interferences. Therefore, we assigne memcached exclusively to one core on node a to minimize CPU interference while preserving computing resources for PARSEC benchmark.
- Which node does each of the 7 batch jobs run on? Why?
  Answer:
  - blackscholes: node a
  - canneal: node c
  - dedup: node b
  - ferret: node c
  - freqmine: node b
  - radix: node b
  - vips: node c
    * For node a, we assign blackscholes on it. This is because blackscholes, under our experiment, maybe the "easiest" among all these 7 jobs, which is not that sensitive. So we assign it on node a to maximize the parallelization. We also tried just letting memcached running alone on node a but the total running time became longer.
    * For node b, we assign dedup, freqmine and radix on it. Node b is of type n2d-highmem-4, which has 4 cores with higher memory configuration. Because these three jobs are demanding on memory and relatively easy to run compared with canneal or ferret jobs.
    * Node c, which is of type e2-standard-8, has 8 cores and the best configuration. Under our experiment, canneal and ferret are the two most difficult jobs to run.

So we assign canneal, ferret and vips on it, which maximize the parallelization of job running and shorten the total duration.

- Which jobs run concurrently / are colocated? Why?

  Answer:
  As explained in the last subquestion, we assign blackscholes on node a, dedup, freqmine and radix on node b, canneal, ferret and vips on node c.

  - We colocate blackscholes and memcached together, which has been explained in the first subquestion above.
  - We colocate freqmine, dedip and radix together because they are all demanding on memory but also not the most demanding. Also the computational resources required to run them are not significant.
  - Canneal, ferret and vips are colocated together on node c because of three reasons. First, from part2, we know that ferret is very sensitive to l1i, llc and membw interferences, so node c is the best choice for ferret. Second, since node c has 8 cores, we can run the two most hard jobs, canneal and ferret on it. Third, we also want to make use of the free cores after one job is finished, we also assign a medium-hard-level job, vips, on node c.

- In which order did you run the 7 batch jobs? Why?

  Order:

  - On node a: blackscholes
  - On node b: freqmine, dedup, radix
  - On node c: canneal, ferret, vips

  Why:

  - There is only one job on node a, so it is meaningless to discuss the order.
  - For node b and c, we just run the jobs based on the approximation of their running time under our previous experiments. The longer their running time maybe, the earlier we run them. In the following subquestion, we will have a further explaination by incorporating the number of threads allocated for each job.

- How many threads have you used for each of the 7 batch jobs? Why?

  Answer:

  - blackscholes: 1
  - canneal: 2
  - dedup: 2
  - ferret: 6
  - freqmine: 4
  - radix: 2
  - vips: 6

  The threads used for each of the 7 batch jobs are shown above.

  - Node a has 2 cores and we use 1 thread for blackscholes to run, without interfering memcached.

- Node b has 4 cores. We allocate 4 threads for freqmine to make it run faster, because it is the most demanding job compared with other two. Then we equally allocate 2 threads for dedup and radix.
- Node c has 8 cores. We allocate 2 threads for canneal and 6 threads for ferret to make them run faster. Then we also allocate 6 threads for vips because we know ferret will finish soon.

- Which files did you modify or add and in what way? Which Kubernetes features did you use?

  Answer:
  We modify the corresponding *memcache-t1-cpuset*, *parsec-** yaml files. First, we use *taskset* to set CPU affinity for memcached and blackscholes job separately to minimize the interference. Second, we use **nodeSelector**, which is a Kubernetes feature we use, to assign certain job on certain node. For example, if we want to run certain job on *node-a-2core*, we just add *"cca-project-nodetype: 'node-a-2core'"* in its yaml file. Third, we also modify the allocated thread number of each job by the argument "*-n*".

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above):

  Answer:
  We think the above explanation is sufficient.

Please attach your modified/added YAML files, run scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.

Part 4 [74 points]

1. **[18 points]** Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP  \
           --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 5 \
           --scan 5000:125000:5000
```

a) [7 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with T=1 thread, C=1 core
- Memcached with T=1 thread, C=2 cores
- Memcached with T=2 threads, C=1 core
- Memcached with T=2 threads, C=2 cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots:

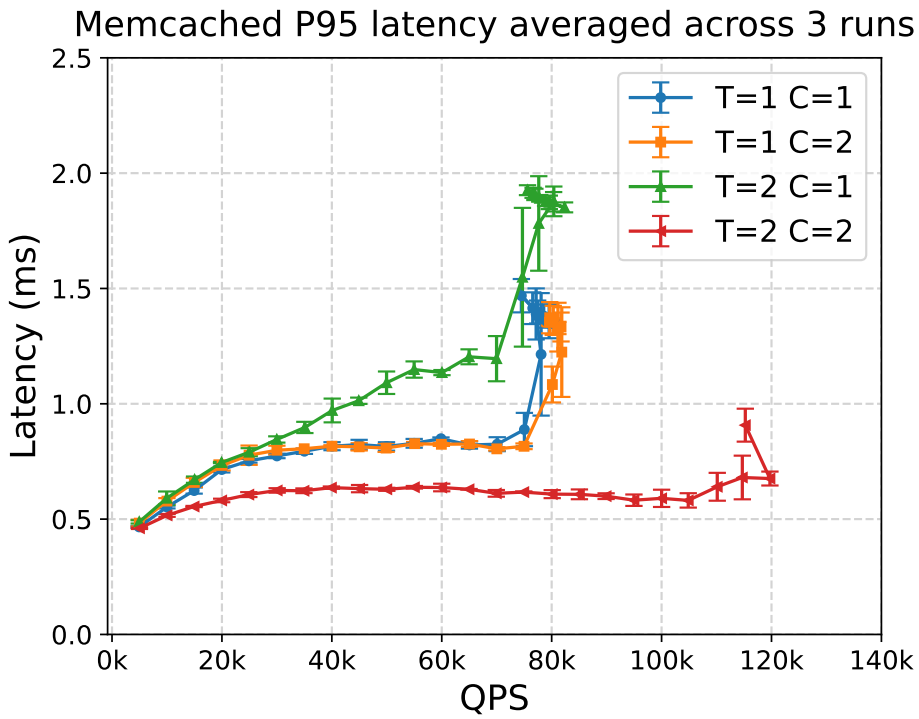


Figure 4: Measurement Result for Part4.1.a

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores.

Summary:
For T=1 C=1, memcached has the highest latency because 2 threads compete with 1 core, QPS can only achieve 80k because of lack of CPU; For T=1 C=1 and T=1 C=2, memcached has similar latency since 1 thread can only utilize 1 core, QPS can only achieve 80k because of lack of CPU; For T=2 C=2 has lowest latency and QPS can achieve 120k because of utilizing 2 CPU cores.

b) [2 points] To support the highest load in the trace (125K QPS) without violating the 1ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer:
We choose 2 threads and 2 cores, since only this can satisfying the 1ms SLO according to figure 4.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 1ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?

Answer:
We choose 2 threads, we can allocate 2 cores to guarantee the SLO when QPS>40k

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$ and $C = 2$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each 5-second load time step.

Plot the performance of memcached using 1-core ($C = 1$) and using 2 cores ($C = 2$) in **two separate graphs**, for $C = 1$ and $C = 2$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 130K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 1ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$ or 200% for $C = 2$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots: Refer to Figure 5 and Figure 6

2. **[17 points]** You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 100K in 10 second time intervals. Use the following command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
           --qps_interval 10 --qps_min 5000 --qps_max 100000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the mcperf measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch

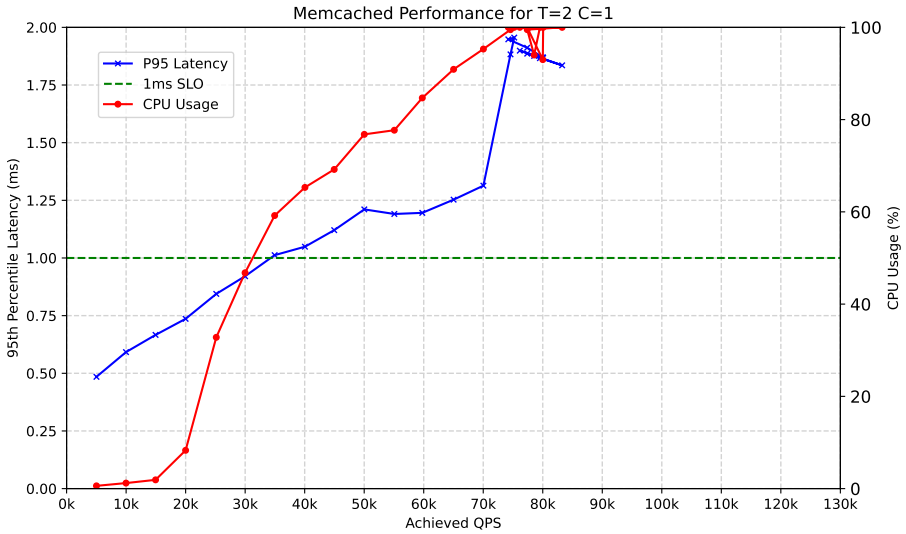


Figure 5: Measurement Result for Part4.1.d with T=2 C=1

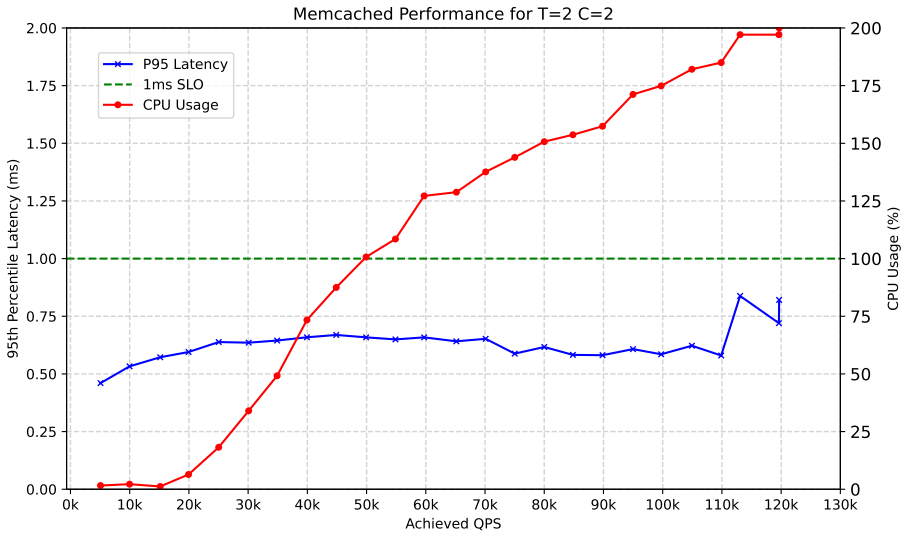


Figure 6: Measurement Result for Part4.1.d with T=2 C=2

jobs as soon as possible without violating the 1ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

  Answer:
  Memcached: dynamic assigned to #0 or #0,1 cores exclusively according to CPU utilization. Batch Jobs: divide into 2 sets, single-core set(blackscholes, canneal, dedup) and multi-core set(ferret, freqmine, radix, vips). Job in single-core set will run on #1 core exclusively when memcached running on #0 core and paused if memcached running on #0,1 core, recording its accumulated running time, the job will run to exit or move to multi-core set when running time reaches a given time threshold. Job in multi-core set will run on #2,3 core exclusively when single-core set unfinished, if single-core set finish, dynamically assign jobs to #1,2,3 core when memcached assigned to #0 core. Scheduler: divide into stage 1: single-core jobs and multi-core jobs stage and stage 2: multi-core jobs. Single-core set total running time is short enough to enter stage 2.

- How do you decide how many cores to dynamically assign to memcached? Why?

  Answer:
  Dynamically assign memcached to #0 or #1 core according based on #0,1 core utilization. We periodically detect interval #0 and #1 CPU utilization when executing jobs. FSM model: state from 0 to n, n is a parameter.

  - Before we execute batch jobs, the state=1, we assign 2 cores for memcached.
  - State 0: memcached is running at #0 core, detect #0 core utilization, if utilization $> \delta_0$, assign memcached to #0,1 core, then move to state 1.
  - State 1...n: memcached is running at #0,1 core, detect #0,1 core total utilization, if utilization $< \delta_1$, move to (state+1) % n, if move to state 0, then assign memcached to #0 core.
  - Parameters setting: $n = 2$; $\delta_0 = 60\%$, $\delta_1 = 40\%$, initially based on Part4.1 then tuning to the these values.
  - After finishing all jobs, assign 2 cores for memcached.

  Reasoning: we run memcached on 2 threads, 2 cores are enough to satisfying 1ms SLO, when detecting low CPU utilization, meaning QPS is low, and assigning memcached to 1 core will not violate SLO. When detecting high CPU utilization, meaning QPS become high, we need to assign 2 core for memcached to guarantee SLO. We switch 2 cores immediately in order to guarantee SLO. And we use more than 1 state for 2 cores, which means when we continuously detect low CPU utilization for #0,1 cores, then we remove 1 core from memcached, in this way, we can reduce jitters, prevent from assigning too frequent between 1 core and 2 cores unnecessarily.

- How do you decide how many cores to assign each batch job? Why?

  Answer:

  - blackscholes: 1-2
    blackscholes has about 12s single thread executing time in beginning which can only utilize 1 core. After 12s, the job scale well for 2 threads, so we move the job from single-core set state to multi-core set state.
  - canneal: 1-2
    canneal has about 38s single thread executing time in beginning which can only utilize 1 core. After 38s, the job scale well for 2 threads, so we move the job from single-core set state to multi-core set state.
  - dedup: 1
    dedup has low scalability according to Part2, so only assign 1 core.
  - ferret: 2
    ferret scale well for 2 threads, but not scale wll for 3 threads according to Part2, so assign 2 cores.
  - freqmine: 2-3
    freqmine scale well for 3 threads, and memcached may use 2 cores or other job in single-core set is running, so dynamically assign 2 or 3 cores for the job.
  - radix: 2
    radix scale well for 2 or more threads, but run to unstop if we assign 3 threads, so we assign 2 cores for the job.
  - vips: 2-3
    vips scale well for 3 threads, and memcached may use 2 cores or other job in single-core set is running, so dynamically assign 2 or 3 cores for the job.

- How many threads do you use for each of the batch job? Why?

  Answer:

  - blackscholes: 2
  - canneal: 2
  - dedup: 1
  - ferret: 2
  - freqmine: 3
  - radix: 2
  - vips: 3

  Reasoning: use potential max assigned cores number of threads to each jobs. We need such number of threads to utilize the CPU cores and use more threads than max potential assigned cores is meaningless.

- Which jobs run concurrently / are collocated and on which cores? Why?

  Answer:
  No job will collocate on the same core, in this way we can reduce cpu, l1d and l1i competition. At anytime, besides memcached, we concurrently run up to 2 jobs to reduce llc and memBW competition. The concurrently running jobs are not fix, and there are some possible concurrently running pairs (blackscholes, ferret), (canneal ferret),

(canneal, radix), (canneal, vips), (dedup, blackscholes), (dedup, canneal), (dedup, vips), (dedup, freqmine). We mainly cosider the llc interference, we never concurrently run dedup and ferret which has highest llc interference. For each pair, there is a job has relatively low llc interference or the pair is only run at relatively short time.

- In which order did you run the batch jobs? Why?

  Order (to be sorted): blackscholes, canneal, dedup, ferret, freqmine, radix, vips

  Why:

  The running order is not fix. An example order:
  ferret, blackscholes(single-core), canneal(single-core), blackscholes(multi-core), radix, dedup, canneal(muti-core),vips freqmine.

  There are other many potential orders, here are the law: For single-core set, the order is blackscholes, canneal, dedup; for multi-core set, the order is ferret, radix, vips, freqmine, and when blackscholes or canneal move to multi-core set, they can preempt vips or freqmine

  Reasoning: the blackscholes run first in single-core set it has low (12s) single thread state can move to multi-core as fast as possible, then canneal, dedup because we set dedup to run to end because of low scalability; ferret run first because we decide to assign it 2 cores, so it should run at the end, also ferret has highest llc interference, we concurrently run it with blackscholes witch has relatively low llc interference. vips and freqmine run at the end and can be preempted by blackscholes and canneal because only after stage 1(all single-core job finished), they can utilize up to 3 cores.

- How does your policy differ from the policy in Part 3? Why?

  Answer:

  Compared to Part 3 static policy, Part 4 policy is much more dynamical, dynamically assign memcached and batch jobs CPU cores, Part4 has more detail observation on jobs, analysed the running pattern of jobs. Generally, part 4 assign less CPU cores and jobs have less thread because of lack of CPU resources. The order of Part 4 is also different since the order in Part 4 is dynamical because we introduce pause and unpause and it affect by random seed. And the concurrent jobs is different because it due to which job runs to end or to stage2, it also affect by random seed.

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.

  Answer:

  - CPU utilization detection: use `psutil.cpu_percent(interval=0.3, percpu=True)` to detect CPU utilization of each cores, this will blocking for interval second and get the CPU utilization between the interval.
  - update memcached cores: use
    ```
    subprocess.run(['bash', '-c', f"pid=$(pidof memcached) && \
                        sudo taskset -a -cp {new_cores} $pid"])
    ```
    call bash to get the pid of the memcached and assigned all threads to new cores list
  - start containers: import docker and get the docker client in python, then call `client.containers.run(...)` to start job
  - check the contain if finishes: first get the container by `client.containers.get(job)` and check `container.attrs['State']['Status']` if the status is "exited"
  - pause and unpause: first check the status of the container, running or pause, then decide to call `container.pause()` or `container.unpause()`

- update container's core: use `container.update(cpuset_cpus=new_cores)`

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above):

  Answer:
  We have said that the jobs in single-core set should finish before the jobs in multi-core set end, so it affect the job selection and parameter of threshold of running time of single-core jobs. For example, the canneal take 38s running only utilizes 1 core when runs only, but if canneal concurrently running with other jobs, we assume the time will be longer, nevertheless, we still set the threshold 40s in order to early move the job to the multi-core set, if we move too late, the freqmine will start earlier and run very long time at 2 cores but it has 3 threads.

3. **[23 points]** Run the following `mcperf` memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
           --qps_interval 10 --qps_min 5000 --qps_max 100000 \
           --qps_seed 3274
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time [2] across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency $>$ 1ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
blackscholes	62.79	0.15
canneal	155.99	2.78
dedup	33.5	2.63
ferret	187.45	1.97
freqmine	246.82	5.24
radix	19.91	0.29
vips	56.55	1.72
total time	663.91	3.98

Answer:
SLO violation ratio:
Run 1: 0, since all data point p95 latency $<$ 1ms

[2] Here, you should only consider the runtime, excluding time spans during which the container is paused.

Run 2: 0, since all data point p95 latency $<$ 1ms
Run 3: 0, since all data point p95 latency $<$ 1ms

Include six plots – two plots for each of the three runs – with the following information. Label the plots as 1A, 1B, 2A, 2B, 3A, and 3B where the number indicates the run and the letter indicates the type of plot (A or B), which we describe below. In all plots, time will be on the x-axis and you should annotate the x-axis to indicate which benchmark (batch) application starts executing at which time. If you pause/unpause any workloads as part of your policy, you should also indicate the timestamps at which jobs are paused and unpaused. All the plots will have have two y-axes. The right y-axis will be QPS. For Plots A, the left y-axis will be the 95th percentile latency. For Plots B, the left y-axis will be the number of CPU cores that your controller allocates to memcached. For the plot, use the colors proposed in this template (you can find them in `main.tex`).

Plots:
Refer to figure 7, figure 8, figure 9

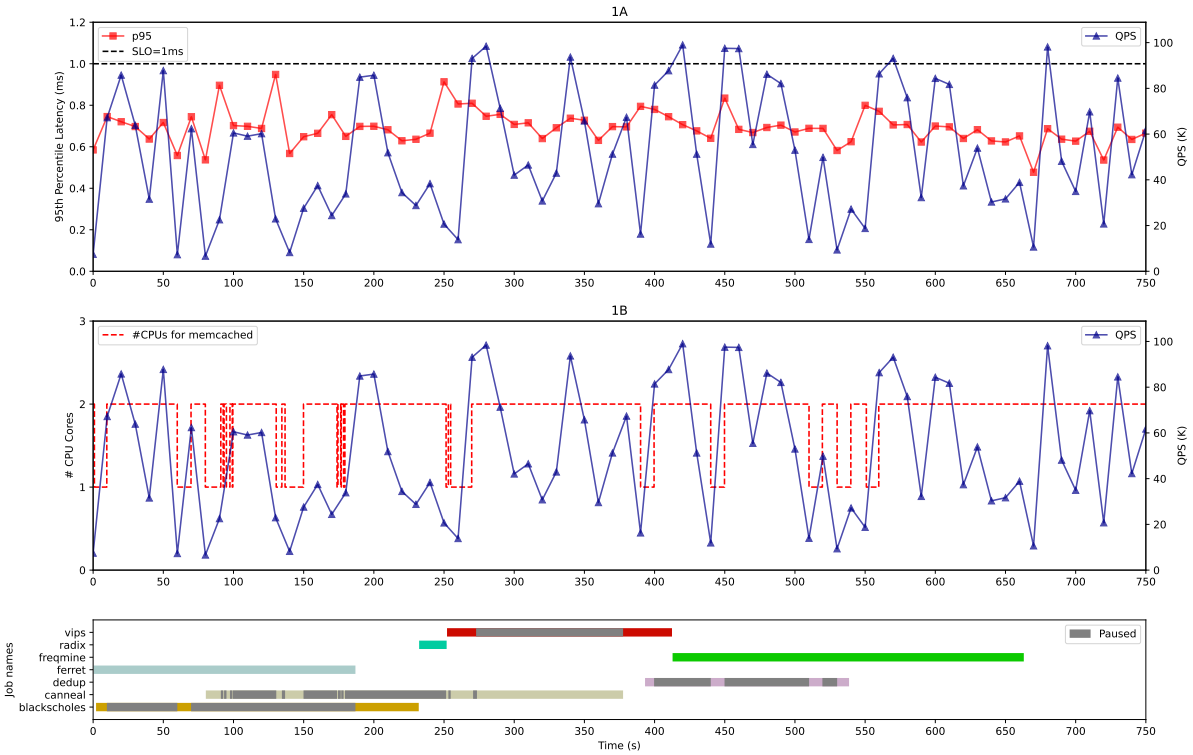


Figure 7: 1A and 1B for Part4.3

4. **[16 points]** Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 10 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
```

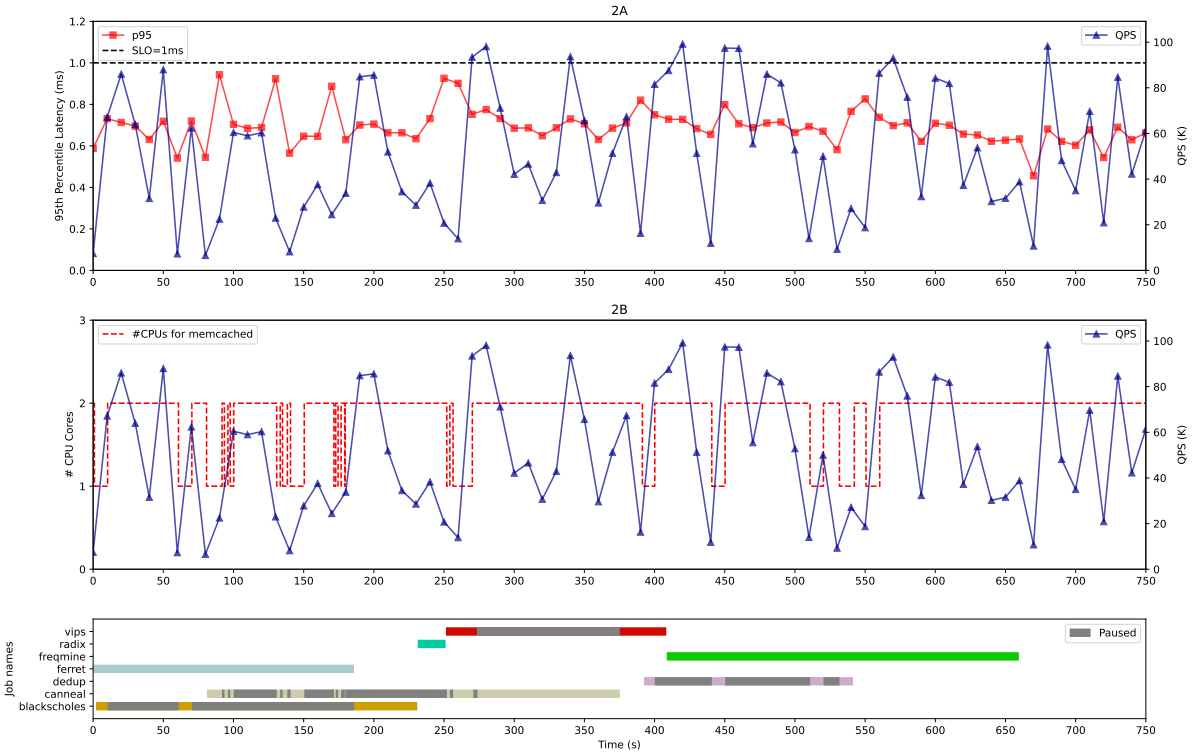


Figure 8: 2A and 2B for Part4.3

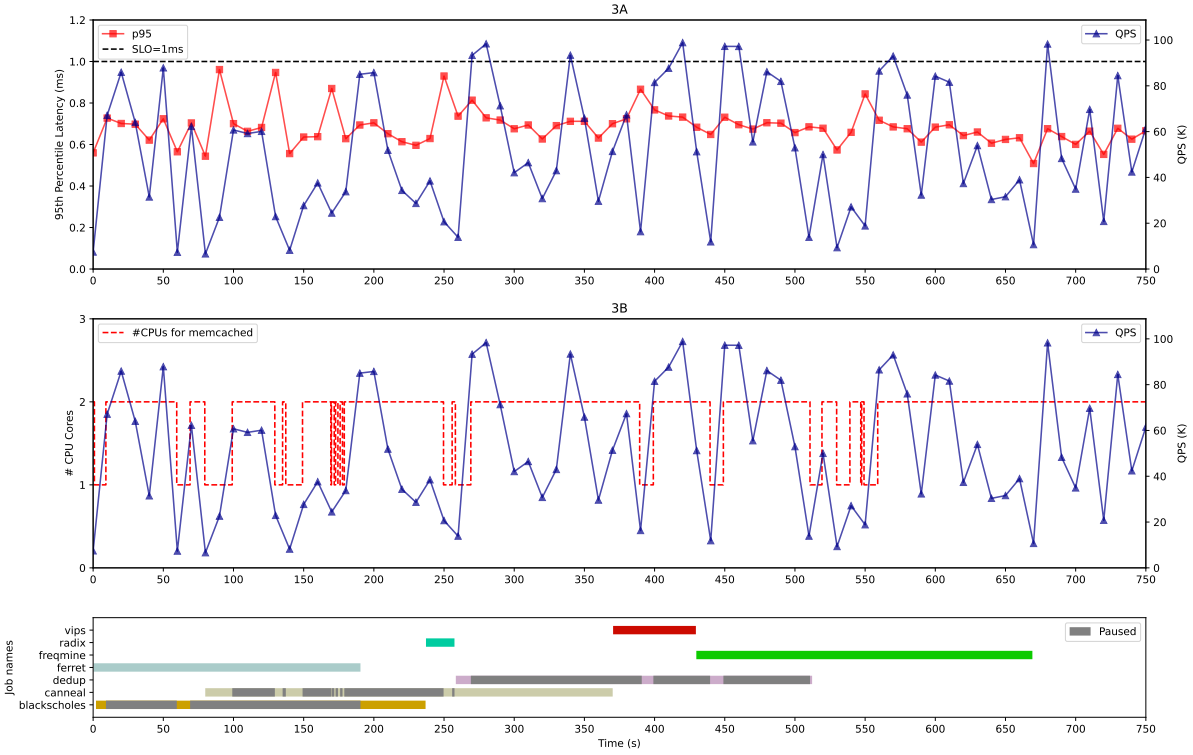


Figure 9: 3A and 3B for Part4.3

```
--qps_interval 5 --qps_min 5000 --qps_max 100000 \
--qps_seed 3274
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Summary:
With the smaller time interval (3s and 4s), we observed more frequent pause, unpause, and update CPU cores more frequently. We also observed very higher peak p95 latency and some data points violate the 1ms SLO. Another observation is it affect our execution orders, the "single-core set" jobs are finished later and now is always canneal preempt freqmine.

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency > 1ms, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

Answer:
Our 5s interval has no data point violate the 1ms SLO, so the violation rate is 0.
Now we reduce the interval to 4s, then observe SLO violation, we calculate the SLO violation rate for the following 3 experiment with `qps_interval=4`
Run 1: 4 out of 170 data points, violation rate = 2.4%
Run 2: 1 out of 167 data points, violation rate = 0.6%
Run 3: 2 out of 162 data points, violation rate = 1.2%

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer:
The smallest `qps_interval` keep SLO violation rate under 3% is 4s, which has been stated above.

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer:
The `psutil.cpu_percent()` recommend the interval at least 0.1s, we use 0.3s interval, in the beginning, we `time.sleep()` 1s or under 1s, but finally find that we didn't need to sleep since the `cpu_percent()` is already blocking the interval time, so when we check the scheduler itself CPU utilization, it still remain under 1%. Aslo, using a more conservative CPU core assignment tactics like reducing the threshold to move memcached from 1 core to 2 cores or increasing the threshold to move from 2 cores to 1 cores. But we did not change such parameters in order to maintain the performance, so with the 4s interval, we only observe the very slight longer total time compared to 10s interval.

Use this `qps_interval` in the command above and collect results for three runs. Include the same types of plots (1A, 1B, 2A, 2B, 3A, 3B) and table as in Question 3.

Plots:
Refer to figure 10, figure 11, figure 12

job name	mean time [s]	std [s]
blackscholes	62.82	0.78
canneal	158.9	10.88
dedup	31.54	0.1
ferret	202.49	3.28
freqmine	223.86	1.73
radix	21.87	0.42
vips	55.32	1.01
total time	664.65	12.07

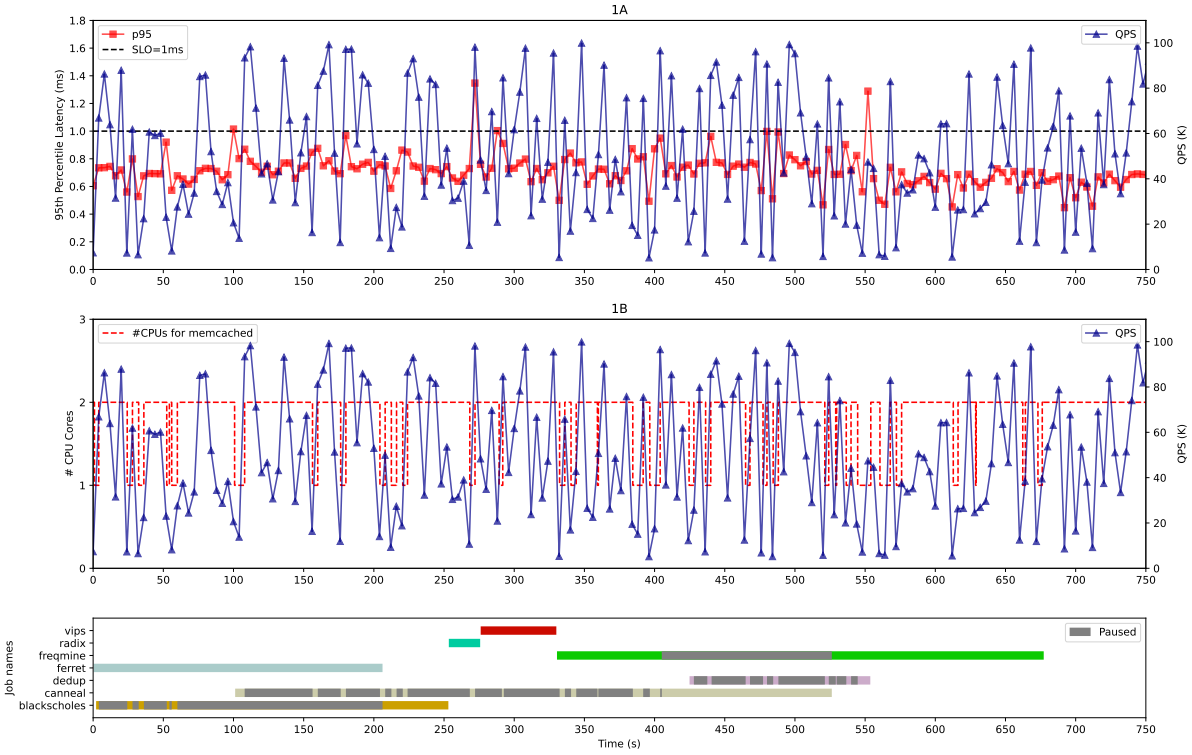


Figure 10: 1A and 1B for Part4.4

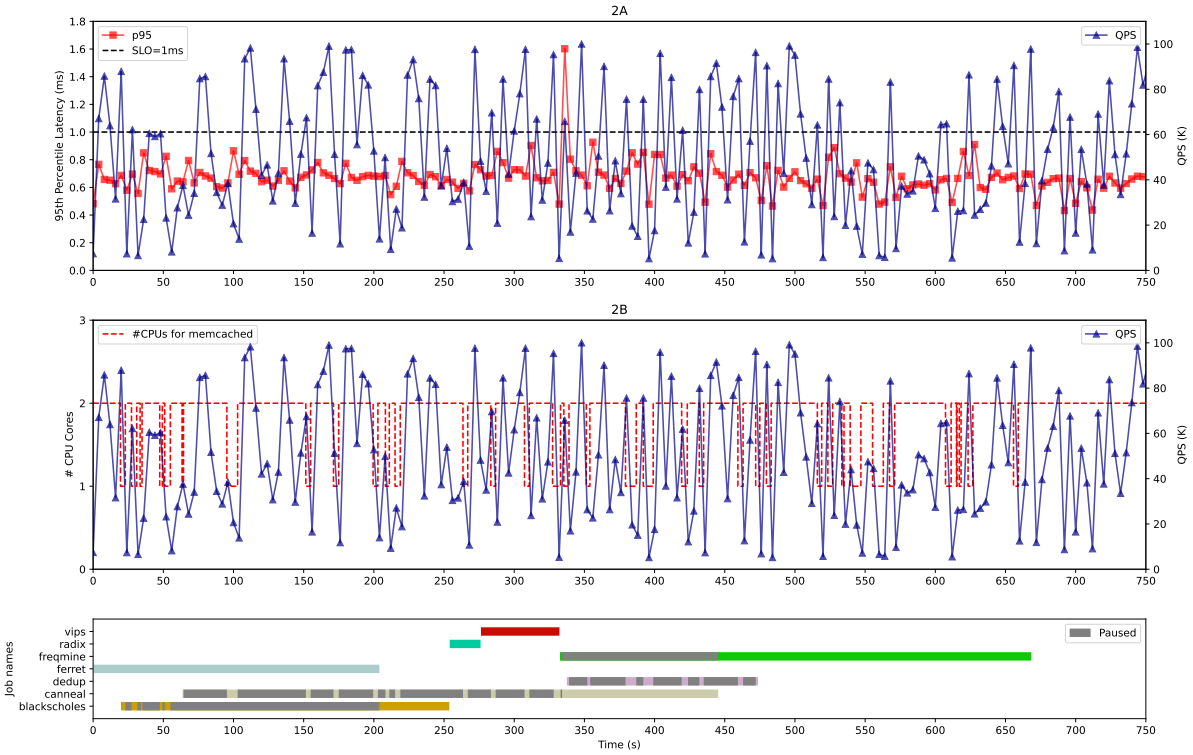


Figure 11: 2A and 2B for Part4.4

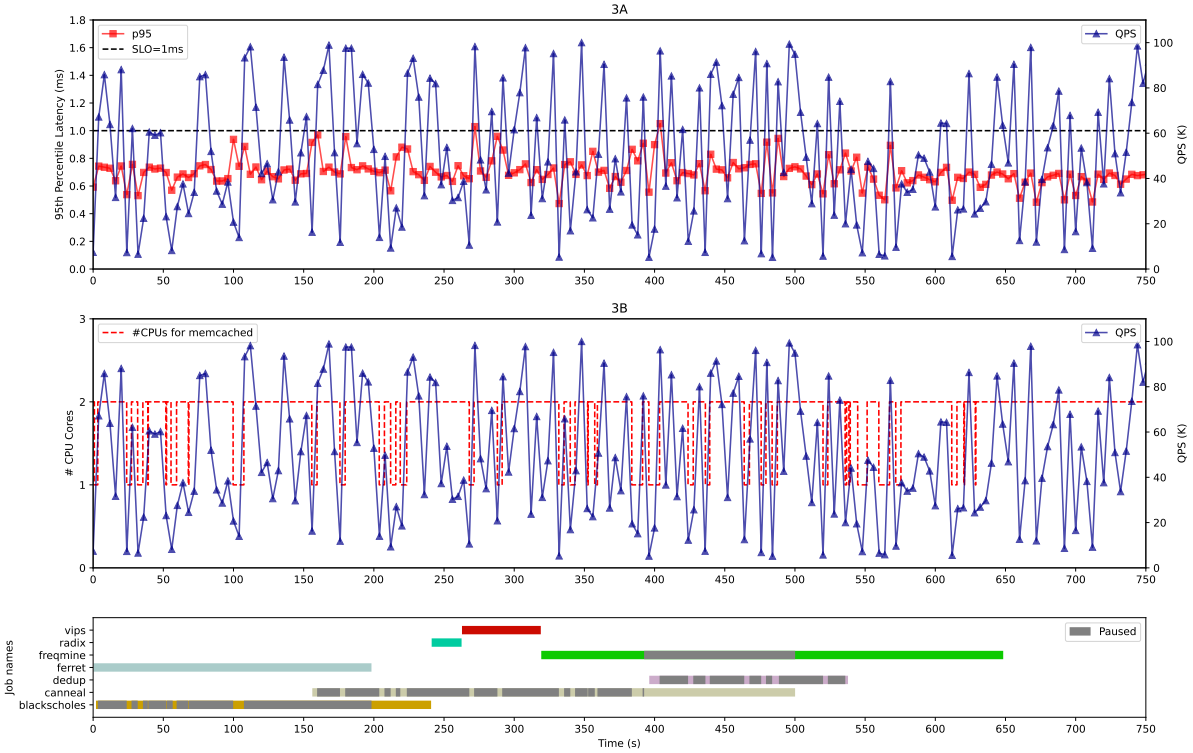


Figure 12: 3A and 3B for Part4.4